Project Proposal: Cache-Aware Graph Reordering for Efficient Graph Neural Network Execution on GPUs

Sahil Murtaza
University of Virginia
Charlottesville, USA
vpn9ej@virginia.edu

Youke Zhang
University of Virginia
Charlottesville, USA
nfz5mv@virginia.edu

Zach Yu
University of Virginia
Charlottesville, USA
rqx6rs@virginia.edu

Albert Liu
University of Virginia
Charlottesville, USA
htr5xa@virginia.edu

Abstract

Graph Neural Networks (GNNs) and Graph Transformers suffer from significant hardware inefficiencies during training due to the irregular and sparse nature of graph data. When aggregating neighbor features, GPUs experience high cache miss rates as threads are forced to fetch data from non-contiguous physical memory locations in High Bandwidth Memory (HBM). This proposal outlines a project to investigate, implement, and benchmark cache-aware node reordering algorithms (such as Reverse Cuthill-McKee and METIS) as a pre-processing step for GNN execution. By systematically transforming the adjacency matrix layout, we aim to maximize spatial locality, increase L2 cache hit rates, and accelerate overall epoch training time without altering the underlying mathematical models or prediction accuracy.

Keywords

Graph Neural Networks, GPU Architecture, Memory Coalescing, Node Reordering, Cache Optimization

1 Introduction: The Problem

The dominant bottleneck in accelerating Graph Neural Networks (GNNs) is the "Memory Wall." Modern GPUs are heavily optimized for dense, contiguous matrix multiplications (GEMMs), which achieve massive speedups through memory coalescing. However, real-world graphs are highly sparse and topologically irregular.

During the message-passing phase of a GNN, a node updates its representation by aggregating features from its local neighborhood. Mathematically, this relies on sparse matrix-vector multiplication (SpMV). A standard update rule at layer l is defined as:

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} W^{(l)} h_j^{(l)}\right) \tag{1}$$

Because the neighbors $j \in \mathcal{N}(i)$ are scattered randomly across the adjacency matrix, GPU threads must fetch memory from non-contiguous physical addresses. This irregularity causes severe cache misses, forcing the compute cores to stall while data is fetched from slower High Bandwidth Memory (HBM) rather than the fast L1/L2 SRAM caches.

This project aims to solve this by optimizing the memory layout of graph data prior to training. By computing a permutation matrix P to reassign node IDs, we can cluster connected nodes closer together in memory, making GNN computations fundamentally more cache-friendly.

2 Existing Literature

The intersection of graph layout optimization and deep learning systems is a highly active research area. This project builds upon several foundational and contemporary works:

- **Rabbit Order (Arai et al., 2016) [1]:** Introduced hierarchical, community-based node reordering to map graph topologies to underlying cache hierarchies.
- **GNNAdvisor (Wang et al., 2021) [3]:** A seminal systems paper introducing 2D workload management and lightweight node renumbering to improve spatial locality for GNNs on NVIDIA GPUs.
- **Empirical Graph Reordering (Bayer et al., 2024) [2]:** A recent study evaluating diverse reordering strategies on PyTorch Geometric, proving that lightweight algorithmic reordering effectively reduces GPU-based training time.

3 Methodology

3.1 Datasets

We will utilize medium-to-large-scale datasets from the Open Graph Benchmark (OGB), specifically designed to expose hardware memory bottlenecks:

- **`ogbn-arxiv`:** A paper citation network comprising 169K nodes and 1.1M edges.
- **`ogbn-products`:** An Amazon co-purchasing network comprising 2.4M nodes and 61.8M edges.

Data will be programmatically acquired and parsed using the official `torch_geometric.datasets` API.

3.2 Algorithms and Techniques

A. Workload Models: We will benchmark reordering techniques across two standard architectures implemented in PyTorch Geometric:

(1) *GraphSAGE*: To profile memory accesses during neighbor sampling.
(2) *Graph Attention Network (GAT)*: To profile memory accesses during dense attention coefficient calculations.

B. Node Reordering Algorithms: An offline pre-processing pipeline will apply permutation matrices to the graph's adjacency and feature matrices using:

- **Reverse Cuthill-McKee (RCM):** Implemented via `scipy.sparse.csgraph`. This algorithm minimizes the bandwidth of the sparse adjacency matrix, clustering non-zero elements strictly near the diagonal.
- **METIS Community Partitioning:** Using the `pymetis` library to identify dense subgraphs and renumber nodes sequentially within those communities, maximizing spatial locality.

3.3 Hardware Profiling

To prove the root cause of any observed speedups, we will utilize **NVIDIA Nsight Compute (`ncu`)**. Custom bash scripts will profile the compiled CUDA kernels during PyTorch execution to extract hardware-level L2 Cache metrics.

4 Evaluation Plan

The project will be evaluated quantitatively through a rigorous ablation study comparing baseline graphs (random/default ordering) against reordered graphs.

- **Primary Systems Metric:** Epoch Training Time (milliseconds/epoch). We target a 15–30% latency reduction.
- **Secondary Hardware Metric:** L2 Cache Hit Rate and DRAM Read/Write throughput (via `ncu`).
- **Machine Learning Sanity Check:** Node Classification Accuracy. This must remain mathematically identical to the baseline to prove the layout transformation is lossless.

5 Timeline and Expected Outcomes

Assuming an 8-week timeline:

- **Weeks 1–2:** Environment setup, dataset acquisition, baseline model implementation, and initial timing benchmarks.
- **Weeks 3–4:** Pipeline development. Implementation of RCM and METIS algorithms and validation of PyG tensor compatibility.
- **Weeks 5–6:** Execution and profiling. Running models on reordered datasets and extracting `ncu` metrics.
- **Weeks 7–8:** Analysis, data visualization (adjacency matrix sparsity patterns), and final report generation.

Expected Outcomes: (1) A reusable PyTorch pipeline for automatic graph cache-optimization. (2) A comprehensive benchmarking report detailing the exact relationship between graph layout, GPU L2 cache hit rates, and GNN training latency.

References

[1] Junya Arai, Hiroaki Shiokawa, Takeshi Yamamuro, Makoto Onizuka, and Sotetsu Imai. 2016. Rabbit Order: Just-in-Time Parallel Reordering for Fast Graph Analysis. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 22–31. doi:10.1109/IPDPS.2016.110

[2] Moritz Bayer et al. 2024. Can Graph Reordering Speed Up Graph Neural Network Training? *arXiv preprint arXiv:2409.xxxxx* (2024).

[3] Yuke Wang, Boyuan Feng, Gushu Li, Shuangchen Li, Lei Deng, Yuan Xie, and Yufei Ding. 2021. GNNAdvisor: An Adaptive and Efficient Runtime System for GNN Acceleration on GPUs. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. 515–531.